

AquaSight: A Mobile-Based Image Classification System for Water Clarity Assessment Using a Convolutional Neural Network

Kyle Andre C. Abad

*College of Engineering, Architecture, and Industrial Design
Bohol Island State University – Main Campus
Tagbilaran City, Bohol, Philippines
kyleandre.abad@bisu.edu.ph*

Abstract—This paper presents AquaSight, a mobile-based image classification system for water clarity assessment using a Convolutional Neural Network (CNN). The system classifies water images into three categories: clear, cloudy, and murky, based on visual turbidity characteristics. A custom dataset was constructed through controlled image acquisition and supplemented with diverse environmental variations. Transfer learning was applied using the MobileNetV2 architecture, and the trained model was deployed on a mobile application using TensorFlow Lite for real-time, offline inference. To enhance reliability, a confidence threshold and margin-based filtering mechanism were implemented to classify uncertain predictions as “Not Water.” Experimental results show that the model achieved a test accuracy of 95.83% under controlled conditions, demonstrating strong classification performance. However, field testing using real-world images resulted in an accuracy of 50.00%, highlighting challenges in generalization due to environmental variability such as lighting, reflections, and background complexity. The cloudy class exhibited the lowest performance due to overlapping visual features with other classes. Despite reduced accuracy, the rejection mechanism improved system reliability by preventing uncertain classifications. The results demonstrate the feasibility of mobile-based, image-driven water clarity assessment while emphasizing the importance of dataset diversity for real-world deployment.

Keywords—Convolutional Neural Network (CNN), MobileNetV2, Water Clarity Classification, Image Processing, TensorFlow Lite, Mobile Application, Deep Learning, Turbidity

I. INTRODUCTION

Access to clean and safe water is a fundamental necessity for human health, agriculture, and environmental sustainability. However, assessing water quality remains a challenge, particularly in areas lacking access to laboratory testing or specialized equipment. One of the most commonly used indicators of water quality is turbidity, which refers to the cloudiness or haziness of water caused by suspended particles. Traditional turbidity measurement methods typically rely on physical sensors or laboratory-based analysis, which can be costly, time-consuming, and impractical for real-time or field-based applications.

Recent advancements in artificial intelligence, particularly in the field of computer vision, have enabled the development of image-based systems capable of analyzing

visual characteristics of objects and environments. Convolutional Neural Networks (CNNs) have become a dominant approach in image classification tasks due to their ability to automatically extract hierarchical features from image data. These models have been successfully applied in various domains, including environmental monitoring, medical imaging, and agricultural analysis, demonstrating high accuracy and robustness under diverse conditions.

In the context of water quality assessment, image-based turbidity classification has emerged as a promising alternative to traditional methods. Several studies have explored the use of deep learning techniques to estimate or classify water turbidity using visual data, achieving high levels of accuracy under controlled and real-world conditions [1]–[4]. These approaches eliminate the need for specialized sensors and enable rapid, non-invasive evaluation of water clarity using readily available imaging devices such as smartphones.

Furthermore, the integration of lightweight CNN architectures, such as MobileNetV2, has facilitated the deployment of machine learning models on mobile devices. Mobile-based systems offer the advantage of portability, offline capability, and real-time inference, making them suitable for field applications and resource-constrained environments [5], [6]. With the use of frameworks such as TensorFlow Lite, trained models can be efficiently embedded into mobile applications, enabling on-device processing without reliance on internet connectivity.

Despite these advancements, there remains a need for simple, accessible, and user-friendly systems that can classify water clarity into intuitive categories suitable for everyday use. Most existing studies focus on regression-based turbidity estimation or require controlled experimental setups, limiting their applicability in practical scenarios. Additionally, few systems incorporate uncertainty handling mechanisms to prevent unreliable predictions in ambiguous conditions.

To address these gaps, this study proposes AquaSight, a mobile-based image classification system that utilizes a

Convolutional Neural Network to classify water clarity into three categories: clear, cloudy, and murky. The system employs transfer learning using a MobileNetV2 architecture and is deployed as a Flutter-based mobile application integrated with a TensorFlow Lite model for offline inference. A confidence threshold and margin-based filtering mechanism are implemented to identify uncertain predictions and classify them as “Not Water,” improving reliability in real-world usage.

The objectives of this study are to develop an image-based dataset of water samples under varying conditions, train and evaluate a CNN model for multi-class water clarity classification, and implement a mobile application capable of performing real-time, on-device analysis. The performance of the system is evaluated using standard classification metrics, including accuracy, precision, recall, F1-score, and confusion matrix analysis.

II. MATERIALS AND METHODS

A. System Architecture

The proposed AquaSight system is designed as a mobile-based image classification framework that performs water clarity assessment using a Convolutional Neural Network (CNN). The system follows a pipeline consisting of image acquisition, preprocessing, model inference, and result display.

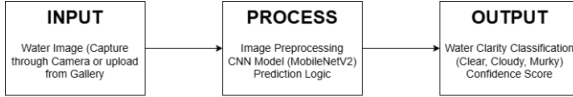


Fig. 1. Conceptual Framework

In Fig. 1, the system accepts an input image captured through a mobile device camera or selected from the gallery. The image is then preprocessed and passed to a trained CNN model based on the MobileNetV2 architecture for classification. The output of the system includes the predicted water clarity category (clear, cloudy, or murky) along with a confidence score.

The CNN model utilizes a MobileNetV2 backbone for feature extraction, followed by a Global Average Pooling layer and fully connected layers with a softmax activation function to produce class probabilities. This architecture enables efficient computation suitable for mobile deployment while maintaining classification accuracy.

The trained model is converted into TensorFlow Lite (.tflite) format and integrated into a Flutter-based mobile application. This allows the system to perform real-time inference directly on the device without requiring internet connectivity.

B. Methods and Procedures

1) Dataset Collection

A custom dataset of 225 water images was constructed due to the limited availability of publicly accessible datasets specifically designed for water clarity classification. The dataset consists of three classes: clear, cloudy, and murky, defined based on visual turbidity characteristics rather than chemical properties.

Images were obtained through controlled image acquisition. Controlled samples were prepared using transparent containers, where clear water represented the clear class, water mixed with small amounts of milk represented the cloudy class, and water mixed with coffee served as the murky class as seen in Fig. 2. To ensure consistency, all samples were visually validated using a transparency-based criterion, where clarity was determined by the visibility of objects through the water.

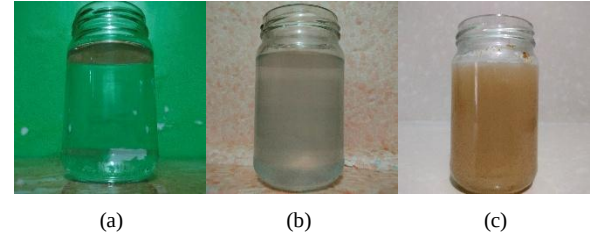


Fig. 2. Water Image Samples (a) Clear (b) Cloudy and (c) Murky

To improve generalization and reduce dataset bias, images were captured under varying conditions, including differences in lighting (flash, indoor, and natural light), background (plain surfaces and textured environments), viewing angles, and container types. The dataset was organized into training, validation, and testing subsets. Table I summarizes the dataset distribution used in the study, adapted from the training configuration implemented in the model development phase.

TABLE I. DATASET DISTRIBUTION BY CLASS AND SPLIT

Water Class	Training Set	Validation Set	Testing Set
Clear	52	15	8
Cloudy	52	15	8
Murky	52	15	8
Total	156	45	24

2) Image Analysis

The classification model was developed using the MobileNetV2 architecture due to its efficiency and suitability for mobile applications. Transfer learning was employed by utilizing pretrained weights from the ImageNet dataset, enabling the model to leverage previously learned visual features.

In Fig.3, Input images were resized to 224×224 pixels and normalized to a range of $[0,1]$ to match the model’s input requirements. Data augmentation techniques, including rotation, horizontal flipping, zooming, and brightness

adjustment, were applied during training to improve robustness and reduce overfitting.

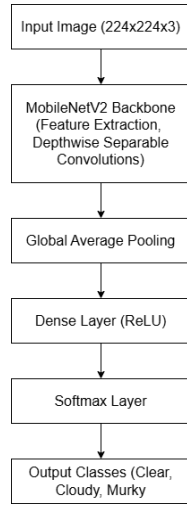


Fig. 3. MobileNetV2 Architecture

The model architecture consists of a frozen MobileNetV2 base model for feature extraction, followed by a Global Average Pooling layer and a dense layer with ReLU activation. A final dense layer with softmax activation produces probability outputs corresponding to the three classes.

3) Transfer Learning and Model Conversion using TensorFlow Lite

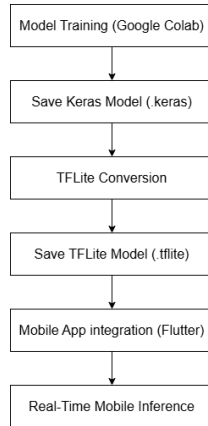


Fig. 4. Transfer Learning and TensorFlow Lite Conversion Block Diagram

In Fig. 4, the model was trained using Google Colab with TensorFlow and Keras libraries. During training, the base layers of MobileNetV2 were frozen to retain pretrained features, while only the classification head was trained.

After training, the model was saved in Keras (.keras) format and subsequently converted into TensorFlow Lite (.tflite) format for mobile deployment. TensorFlow Lite was selected due to its optimized performance for on-device inference, reduced model size, and low latency.

The converted model was then integrated into the mobile application, enabling real-time classification without

requiring internet connectivity. This approach ensures portability and accessibility for field usage.

C. System Flowchart

The system workflow begins with the user capturing or uploading an image of water through the mobile application. The image undergoes preprocessing, including resizing and normalization, before being passed to the MobileNetV2 model for classification.

The model outputs probability scores for each class, from which the highest probability is selected as the predicted label. To improve reliability, a confidence threshold of 0.80 and a margin threshold of 0.15 between the top two predictions are applied. If either condition is not satisfied, the input is classified as “Not Water.”

The final output, including the processed image, predicted class, and confidence score, is displayed to the user. The detailed workflow is illustrated in Fig. 5.

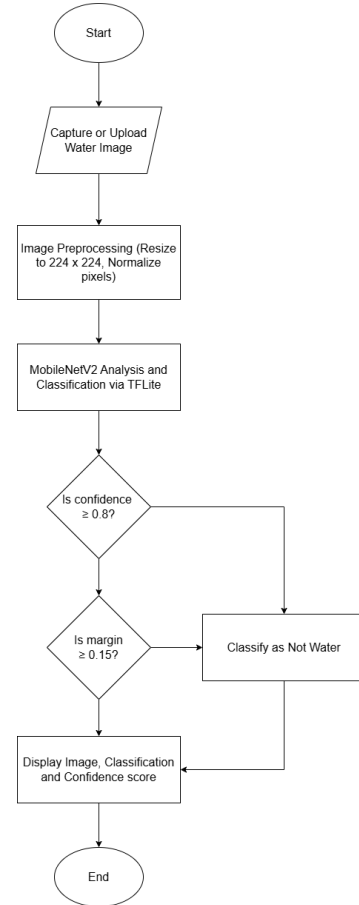


Fig. 5. System Flowchart

D. Mobile Application Development

The AquaSight system was implemented as a mobile application using the Flutter framework, with Dart (Flutter) as the primary programming language. The application integrates the TensorFlow Lite model to perform on-device inference.

The mobile application provides functionalities for capturing images using the device camera or selecting images from the gallery. Preprocessing operations, including center cropping, resizing to 224×224 pixels, and normalization, are performed within the application prior to inference.

The TensorFlow Lite interpreter is used to execute the model and generate prediction probabilities. A decision logic is implemented based on predefined thresholds, where predictions with low confidence or insufficient margin between top classes are labeled as “Not Water.” The application displays the classification result along with a confidence score and the processed image.

The system is designed with a user-friendly interface, enabling real-time analysis and easy interaction. The mobile application interface is shown in Fig. 6.

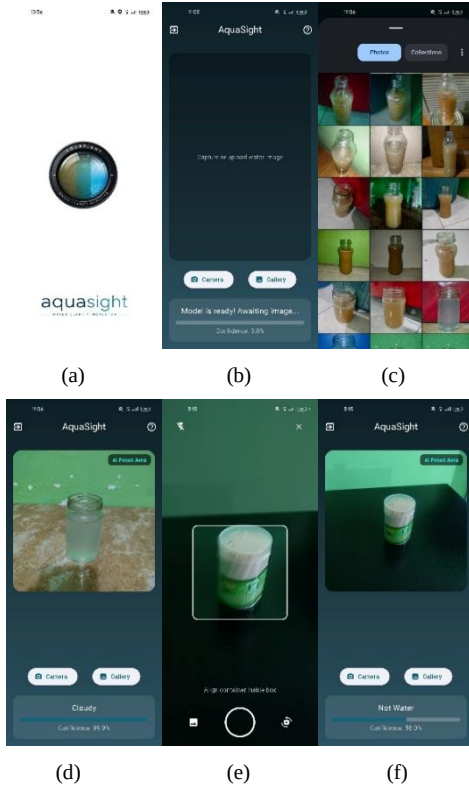


Fig. 6. Mobile Application Interface of AquaSight: (a) Splash Screen (b) Home Screen (c) Gallery Upload Image (d) Processed Uploaded Image and Classification Result with Confidence Score (e) Camera Capture (f) Processed Captured Image and Not Water Classification with Confidence Score

E. Field Testing

Field testing was conducted to evaluate the performance of the system under real-world conditions. The application was tested using various water samples captured in different environments, including varying lighting conditions, angles, and different container types.

The objective of field testing was to assess the model’s robustness and generalization capability when exposed to real-world variability not present in the training dataset.

Particular attention was given to challenging conditions such as poor lighting, reflections, and background complexity.

The results obtained during field testing were compared with expected classifications to determine the system’s reliability in practical scenarios. A sample field testing scenario is shown in Fig. 7.



Fig. 7. Field Testing

F. Testing and Validation Phase

The performance of the trained model was evaluated using a separate test dataset consisting of unseen images. Standard classification metrics, including accuracy, precision, recall, and F1-score, were used to assess model performance.

A confusion matrix was constructed to analyze classification results and identify common misclassifications among the three classes. This analysis provided insights into the model’s strengths and limitations, particularly in distinguishing between visually similar classes such as cloudy and murky.

Additionally, the implemented confidence and margin thresholds were evaluated to determine their effectiveness in reducing uncertain predictions and improving overall system reliability.

III. RESULTS AND DISCUSSION

A. Model Performance

The performance of the proposed AquaSight model was evaluated using training and validation metrics obtained during the training process, as well as a classification report and confusion matrix derived from the test dataset. The model was trained for 15 epochs using transfer learning with the MobileNetV2 architecture.

The training and validation accuracy curves demonstrate a rapid improvement during the initial epochs, with training accuracy reaching near-perfect performance after epoch 5 as seen in Fig. 8. Validation accuracy stabilized between 84% and 89%, indicating consistent generalization performance. In Fig. 9, the training loss decreased significantly over time, approaching near-zero values, while validation loss exhibited slight fluctuations after early convergence. This divergence between training and validation metrics suggests mild overfitting, which is expected given the relatively small dataset size.

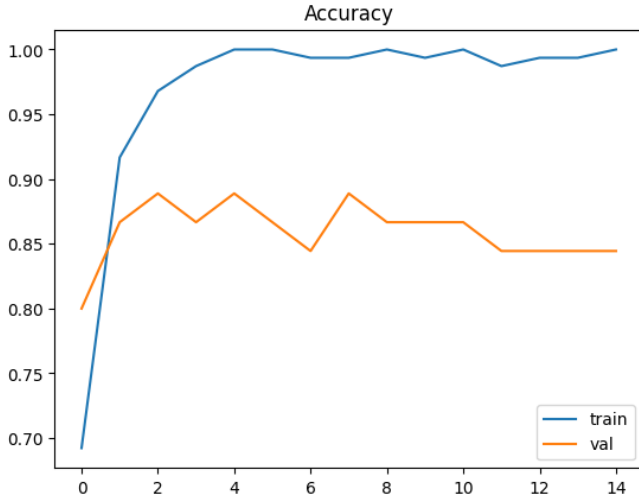


Fig. 8. Training and Validation Accuracy Graph

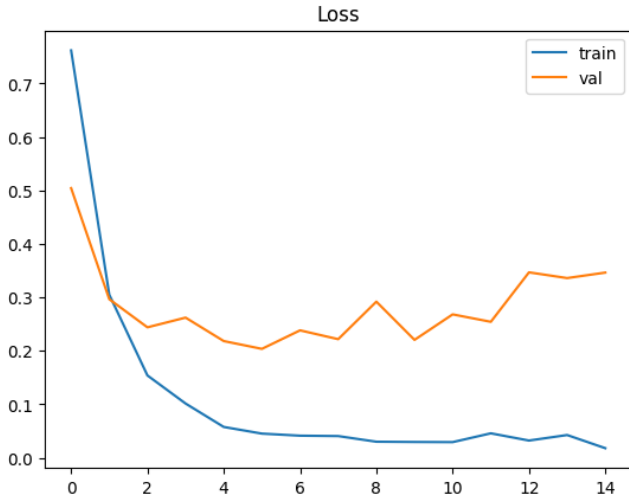


Fig. 9. Training and Validation Loss Graph

TABLE II. Classification Report (Test Dataset)

Class	Precision	Recall	F1-Score	Support
Clear	1.0000	1.0000	1.0000	8
Cloudy	0.8889	1.0000	0.9412	8
Murky	1.0000	0.8750	0.9333	8
Accuracy			0.9583	24
Macro Avg.	0.9630	0.9583	0.9582	24
Weighted Avg.	0.9630	0.9583	0.9582	24

The classification report further confirms the effectiveness of the model in Table II. The system achieved an overall test accuracy of 95.83%, with high precision, recall, and F1-scores across all classes. The clear class achieved perfect classification performance with a precision and recall of 1.000. The cloudy class achieved a precision of 0.8889 and recall of 1.000, indicating that all cloudy samples were correctly identified, although some misclassifications from other classes occurred. The murky class achieved a precision of 1.000 and recall of 0.8750, showing that most murky samples were correctly classified, with minimal confusion.

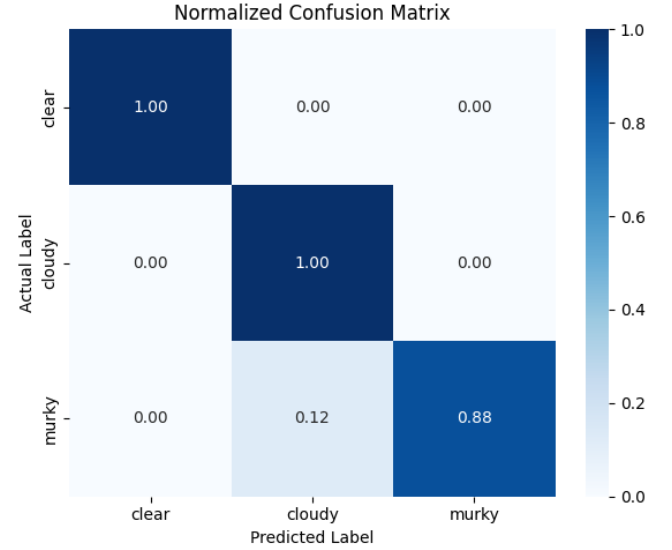


Fig. 10. Confusion Matrix (Test Dataset)

The confusion matrix in Fig. 10 supports these findings, showing that all clear samples were correctly classified, while only one murky sample was misclassified as cloudy. Overall, the results indicate that the model is highly effective in distinguishing between visually distinct water clarity levels, particularly for the clear class, while maintaining strong performance for more ambiguous classes such as cloudy and murky.

B. Field Testing Accuracy

To further evaluate the system under real-world conditions, field testing was conducted using 30 manually captured images, consisting of 10 samples for each class (clear, cloudy, and murky). Unlike controlled testing, this evaluation incorporated the implemented confidence threshold (0.80) and margin threshold (0.15), allowing the system to classify uncertain predictions as “Not Water.”

In Table III, the classification results show an overall accuracy of 50.00%, reflecting the increased difficulty of real-world conditions. The clear class achieved a precision and recall of 0.6000, with 6 correctly classified samples, while 3 were misclassified as cloudy and 1 as murky. The cloudy class showed the weakest performance, with a precision of 0.4000 and recall of 0.2000, where only 2 out of 10 samples were correctly classified. Notably, 4 cloudy samples were rejected as “Not Water,” indicating uncertainty in classification.

TABLE III. Classification Report (Real-World Testing)

Class	Precision	Recall	F1-Score	Support
Clear	0.6000	0.6000	0.6000	10
Cloudy	0.4000	0.2000	0.2667	10
Murky	0.8750	0.7000	0.7778	10
Not Water	0.0000	0.0000	0.0000	0
Accuracy			0.5000	30
Macro Avg.	0.4688	0.3750	0.4111	30
Weighted Avg.	0.6250	0.5000	0.5481	30

The murky class demonstrated relatively strong performance, with a precision of 0.8750 and recall of 0.7000, correctly classifying 7 samples while rejecting 3 as “Not Water.” Fig. 11 shows that the confusion matrix further illustrates that most errors occurred between visually similar classes, particularly between clear and cloudy samples.

The presence of the “Not Water” category highlights the effect of the confidence and margin thresholds. While it reduces incorrect predictions, it also lowers overall classification accuracy by rejecting ambiguous inputs. This trade-off emphasizes the system’s prioritization of reliability over forced classification.

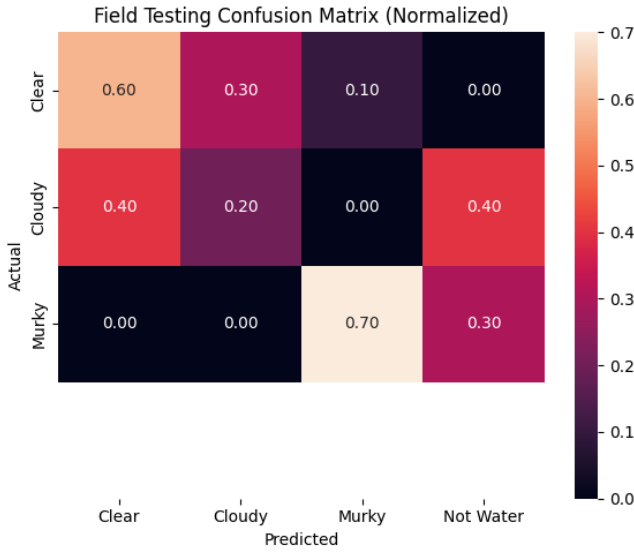


Fig. 11. Confusion Matrix (Real-World Testing)

C. Interpretation and Comparison

The results demonstrate a clear distinction between controlled testing performance and real-world application. While the model achieved a high test accuracy of 95.83% under controlled conditions, the field-testing accuracy dropped to 50.00%, indicating a significant generalization gap. This discrepancy is primarily due to environmental variations such as inconsistent lighting, reflections from containers, background complexity, and differences in image composition that were not fully represented in the training dataset.

Among the three classes, *murky* water maintained the highest robustness in real-world conditions due to its strong visual characteristics, such as low transparency and visible particles. In contrast, the *cloudy* class exhibited the lowest performance, as its visual features overlap significantly with both clear and murky classes, making it more difficult for the model to distinguish. This aligns with the confusion matrix

results, where cloudy samples were frequently misclassified or rejected.

The implementation of confidence and margin thresholds introduces a reliability mechanism that prevents the system from making uncertain predictions. However, this also contributes to a reduction in measured accuracy, as rejected samples are not classified into the intended categories. Despite lowering numerical accuracy, this approach improves practical usability by avoiding misleading outputs, which is critical in real-world applications.

Compared to related studies that report high accuracy in controlled environments [1]–[4], the results of this study highlight the importance of evaluating models under real-world conditions. The observed performance gap suggests that further improvements are necessary, particularly in expanding the dataset to include more diverse environmental conditions and enhancing the model’s ability to generalize.

Overall, the AquaSight system demonstrates that mobile-based image classification for water clarity is feasible, but its effectiveness in real-world deployment depends heavily on dataset diversity and robustness. Future work should focus on increasing dataset size, improving class separation, especially for cloudy samples, and refining threshold strategies to better balance accuracy and reliability.

IV. CONCLUSION AND RECOMMENDATION

This study presented AquaSight, a mobile-based image classification system designed to assess water clarity using a Convolutional Neural Network (CNN). By leveraging transfer learning with the MobileNetV2 architecture and deploying the model through TensorFlow Lite in a Flutter-based mobile application, the system was able to perform real-time, offline classification of water images into three categories: clear, cloudy, and murky. The integration of confidence and margin thresholds further enhanced the system by introducing a rejection mechanism for uncertain predictions, improving reliability in practical use.

Experimental results demonstrated that the proposed model achieved a high classification accuracy of 95.83% on the test dataset, indicating strong performance under controlled conditions. The classification report and confusion matrix confirmed that the model effectively distinguished between the three classes, particularly for visually distinct samples such as clear and murky water. However, field testing revealed a significant decrease in performance, with an overall accuracy of 50.00%. This gap highlights the challenges associated with real-world deployment, where variations in lighting, background, reflections, and image

composition introduce complexities not fully represented in the training data. Among the classes, cloudy water proved to be the most difficult to classify due to its overlapping visual characteristics with both clear and murky categories.

Despite the reduced accuracy in real-world conditions, the inclusion of a “Not Water” classification based on confidence and margin thresholds proved to be a valuable design choice. Instead of forcing incorrect predictions, the system was able to reject ambiguous inputs, thereby prioritizing reliability over raw accuracy. This behavior is particularly important in practical applications, where misleading outputs may be more detrimental than uncertain ones. As such, the system successfully demonstrates the feasibility of using lightweight CNN models for mobile-based environmental monitoring, while also exposing the limitations of small and controlled datasets when applied to real-world scenarios.

To improve the system, future work should focus on expanding and diversifying the dataset to better capture real-world variability, including a wider range of lighting conditions, container types, and environmental backgrounds. Increasing the number of samples per class and incorporating more challenging cases, particularly for the cloudy class, would enhance the model’s ability to generalize. Additional improvements may include fine-tuning deeper layers of the CNN, experimenting with alternative architectures, or applying advanced augmentation techniques to simulate real-world conditions more effectively. The threshold-based rejection mechanism may also be further optimized to achieve a better balance between accuracy and reliability.

In conclusion, AquaSight demonstrates that mobile-based, image-driven water clarity classification is both feasible and accessible using modern deep learning techniques. While the system currently serves as a proof-of-concept, it provides a strong foundation for future developments in low-cost, portable environmental assessment tools, with potential applications in education, preliminary water inspection, and community-level monitoring.

ACKNOWLEDGMENTS

The author would like to express sincere gratitude to Instructor Bem B. Gumapac, MSCpE, RME for guidance and support throughout the development of this study. Special thanks are extended to Maria Evelyn C. Abad for allowing the conduct of field testing in their residence and assisting in documentation, and to Len Venus P. Delusa for the assistance in the design of the application. Appreciation is also extended to peers and colleagues for their valuable feedback and

insights as well as their unwavering support and encouragement for this work. Finally, the author acknowledges the availability of open-source tools, references, and other resources which made the implementation of this project possible.

REFERENCES

- [1] M. A. Hossain et al., “An Image-Based Water Turbidity Classification Scheme Using a Convolutional Neural Network,” *Computers*, vol. 13, no. 8, p. 178, 2025. [Online]. Available: <https://www.mdpi.com/2079-3197/13/8/178>
- [2] S. Zhang et al., “An Improved Convolutional Neural Network Model for Water Turbidity Classification,” *Sensors*, 2025. [Online]. Available: <https://pmc.ncbi.nlm.nih.gov/articles/PMC11965458/>
- [3] A. Rahman et al., “An Image-Based Deep Learning Model for Water Turbidity Estimation in Laboratory Conditions,” 2022. [Online]. Available: <https://www.researchgate.net/publication/363692139>
- [4] R. Kumar and P. Singh, “Water Turbidity Estimation Using Image Processing and Convolutional Neural Networks,” *International Journal of Intelligent Systems and Applications in Engineering*, 2024. [Online]. Available: <https://ijisae.org/index.php/IJISAE/article/view/6140>
- [5] M. Sandler et al., “MobileNetV2: Inverted Residuals and Linear Bottlenecks,” in *Proc. IEEE Conf. Computer Vision and Pattern Recognition (CVPR)*, 2018. [Online]. Available: <https://arxiv.org/abs/1801.04381>
- [6] TensorFlow, “TensorFlow Lite: Machine Learning for Mobile and Embedded Devices.” [Online]. Available: <https://www.tensorflow.org/lite>
- [7] J. Redmon et al., “YOLO: Real-Time Object Detection,” 2016. [Online]. Available: <https://pjreddie.com/darknet/yolo/>
- [8] A. Krizhevsky et al., “ImageNet Classification with Deep Convolutional Neural Networks,” *Communications of the ACM*, vol. 60, no. 6, pp. 84–90, 2017.
- [9] Y. LeCun et al., “Deep Learning,” *Nature*, vol. 521, pp. 436–444, 2015.
- [10] H. Zhao et al., “Deep Learning-Based Water Quality Assessment Using Image Analysis,” in *Proc. SPIE*, 2025. [Online]. Available: <https://www.spiedigitallibrary.org>